

What I learned

Two things I did not know when I opened the packet. For each: the situation, what I did not know, how I learned it, and where it ended up in the system.

1. PUTTING A CRM BEHIND AN AGENT THROUGH MCP

Situation. The CEO note names HubSpot as the system of record. A regulation change reaches many accounts, not one, so the agent had to ask the CRM "who is in scope" and get back a list with a reason per company — without the agent ever writing to the CRM.

What I did not know. I had never connected HubSpot to anything. I did not know how a second tool class sits next to my own MCP tools inside one headless agent run, what the connector could and could not do, or what it would cost.

How I learned it. By hitting walls in order. The session I was building in could not see the connector — MCP servers load at session start — so I wrote a self-contained prompt and had a fresh session build the table. That run showed the connector's edges: it creates properties but not property groups, cannot change an existing enumeration, labels booleans True/False. I then probed the headless CLI for the exact tool names (33 of them), allow-listed six read-only ones beside `reign-tools`, and watched the first fan-out: seven companies screened, six excluded with the reason I would have given, one affected — including HubSpot's own sample company, which the agent excluded as "not an ICP account" rather than choking on it. I also watched a quiet check triple in price the moment 33 tool schemas started loading into every run.

Where it ended up. Ten `reign_*` properties on Companies as the contract between CRM and agent; `applies_to` on each trigger naming the segments and the one flag a regulation turns on; a fixed screening order in the skill (CEO exclusion → segment → four ICP gates → regulation flag) so the verdict is a rule, not a reading; a `score` audit record per affected FS account carrying the HubSpot id; and a two-stage agent on the roadmap because I saw the cost, not because I read about it.

2. TESTING A WATCHER WHEN THE WORLD DOES NOT CHANGE

Situation. The system's job is to notice when a regulator changes something. The regulators had not changed anything and would not on my schedule, so I had a detector with nothing to detect.

What I did not know. Where past versions of a Federal Reserve page or an EU regulation live, how to fetch them in a form my tools could compare byte-for-byte against today's page, and whether the changes I would find were real events or archive artifacts.

How I learned it. The Fed's `Last-Modified` headers gave the first clue — the SR 26-2 files had been touched on 4 May, the letter is dated 17 April, and the 2026 index page had changed two days before I started watching. The Wayback Machine then fought back at each step: its availability API rate-limited me; the plain capture URL for the PDF returned Wayback's own HTML wrapper; the timemap endpoint listed every capture; the `id_` flag returned original bytes; some came back gzip-compressed. My first fixture, built from a non-`id_` capture, had rewritten links that all looked new — the agent read the diff, saw the page text was unchanged, and called it noise with the mechanism spelled out. With raw captures and the same link-extraction code the live fetch uses, the comparisons became clean: the May change to the attachment is one word, `real-word` to `real-world`; the September index change is SR 26-6, unrelated to model risk; the EIOPA DORA page gained three non-substantive links since January.

Where it ended up. A test mode with two scenarios. Entering one rebuilds a separate state tree, copies today's snapshots, and replaces the index-page baselines with Wayback link sets from the dates shown; scenario A also deletes SR 26-2's own snapshots, because on 5 April they did not exist and a real first sighting has to look like one. A check then compares today's live pages against a months-old baseline and replays what actually happened — scenario A produced a brief per affected account, scenario B a noise verdict with the raw diff kept. It also settled a design argument: the hash is the detector, the agent is the judge, and neither works alone.